

PART 4 — COMPLETE SQL + JDBC MASTER NOTES

SQL Basic → Advanced + JDBC | Queries • Joins • Transactions • Java DB access

Detailed standalone handbook: Basic → Intermediate → Advanced, with syntax, complete examples, expected output/behavior, debugging, projects, interview and viva preparation.

ROADMAP

- Relational model
- DDL/DML
- SELECT/filter/sort
- Functions/aggregation
- Joins
- UNION/INTERSECT/EXCEPT
- Subqueries/EXISTS
- Constraints
- Schema changes
- Normalization
- Indexes
- Transactions/ACID
- JDBC setup
- Connection
- Statement
- PreparedStatement
- ResultSet
- CRUD
- Transactions
- Projects

1. Relational Database Fundamentals

A relational database stores data in tables with rows and columns. SQL is used to define, retrieve and manipulate that data.

Core terms

Term	Meaning
Database	Collection of related data
Table	Structured rows and columns
Row	One record
Column	Attribute/field
Primary key	Uniquely identifies a row
Foreign key	References another table's key
Constraint	Rule enforced by database
Index	Data structure for faster lookup

Create database/table

```
CREATE DATABASE college;

CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  age INT CHECK (age >= 0)
);
```

2. SQL Retrieval

SELECT retrieves data. Filtering, sorting, grouping and aggregation form the core query language.

SELECT

```
SELECT id, name, email
FROM students;
```

WHERE

```
SELECT *
FROM students
WHERE age >= 18
AND name LIKE 'A%';
```

ORDER BY/LIMIT

```
SELECT *
FROM students
ORDER BY age DESC
LIMIT 10;
```

Aggregate

```
SELECT
  COUNT(*) AS total,
  AVG(age) AS average_age,
  MIN(age) AS youngest,
  MAX(age) AS oldest
FROM students;
```

GROUP BY/HAVING

```
SELECT department, COUNT(*) AS total
FROM students
GROUP BY department
```

```
HAVING COUNT(*) >= 5;
```

3. Joins

Joins combine rows from related tables.

INNER JOIN

```
SELECT s.name, c.name AS course
FROM students s
INNER JOIN enrollments e
    ON e.student_id = s.id
INNER JOIN courses c
    ON c.id = e.course_id;
```

LEFT JOIN

```
SELECT s.name, e.course_id
FROM students s
LEFT JOIN enrollments e
    ON e.student_id = s.id;
```

Join types

Join	Idea
INNER	Only matching rows
LEFT	All left rows + matches
RIGHT	All right rows + matches
FULL	All rows from both sides where supported
CROSS	Cartesian product

4. Set Operations + Subqueries

Set queries combine compatible result sets; subqueries nest queries.

UNION

```
SELECT email FROM students
UNION
SELECT email FROM teachers;
```

UNION ALL

```
SELECT email FROM students
UNION ALL
SELECT email FROM teachers;
```

Subquery

```
SELECT name
FROM students
WHERE age > (
    SELECT AVG(age)
    FROM students
);
```

EXISTS

```
SELECT s.name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM enrollments e
    WHERE e.student_id = s.id
);
```

Key point: INTERSECT and EXCEPT are supported by many SQL systems but exact support and syntax can vary by database engine.

5. INSERT UPDATE DELETE

DML modifies data.

INSERT

```
INSERT INTO students (id, name, email, age)
VALUES (1, 'Aryan', 'a@example.com', 21);
```

UPDATE

```
UPDATE students
SET age = 22
WHERE id = 1;
```

DELETE

```
DELETE FROM students
WHERE id = 1;
```

Key point: Always use an intentional WHERE clause for UPDATE and DELETE. Test the corresponding SELECT first.

6. DDL, Constraints and Schema Changes

DDL defines database objects.

ALTER

```
ALTER TABLE students
ADD COLUMN phone VARCHAR(20);
```

```
ALTER TABLE students
DROP COLUMN phone;
```

DROP/TRUNCATE

```
DROP TABLE students;
```

```
TRUNCATE TABLE students;
```

Constraints

```
CREATE TABLE courses (
    id INT PRIMARY KEY,
    code VARCHAR(20) UNIQUE NOT NULL,
    credits INT CHECK (credits BETWEEN 1 AND 6)
);
```

7. Normalization, Indexes and Transactions

Good relational design reduces redundancy and preserves consistency.

Normalization overview

- 1NF: atomic values and no repeating groups.
- 2NF: 1NF plus no partial dependency on part of a composite key.
- 3NF: 2NF plus no inappropriate transitive dependency.
- Denormalization can sometimes improve read performance but adds consistency cost.

Index

```
CREATE INDEX idx_students_email
ON students(email);
```

Transaction

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;

-- or
ROLLBACK;
```

Key point: Transactions, isolation and locking details vary across database engines. Learn the ACID model and the isolation levels supported by your chosen database.

8. JDBC Fundamentals

JDBC is Java's standard API for communicating with relational databases.

Connection with DriverManager

```
String url = "jdbc:postgresql://localhost:5432/college";
String user = "postgres";
String password = "secret";

Connection con =
    DriverManager.getConnection(url, user, password);
```

Statement

```
Statement stmt = con.createStatement();

ResultSet rs = stmt.executeQuery(
    "SELECT id, name FROM students"
);

while (rs.next()) {
    System.out.println(rs.getInt("id"));
    System.out.println(rs.getString("name"));
}
```

PreparedStatement

```
String sql =
    "SELECT id, name FROM students WHERE email = ?";

PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, email);

ResultSet rs = ps.executeQuery();
```

Key point: Prefer PreparedStatement for parameterized queries. It improves correctness and helps protect against SQL injection.

9. JDBC CRUD and Transactions

Use try-with-resources so JDBC resources are closed reliably.

Insert

```
String sql =
    "INSERT INTO students(name,email) VALUES (?,?)";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, name);
    ps.setString(2, email);
    ps.executeUpdate();
}
```

Update/delete

```
try (PreparedStatement ps =
    con.prepareStatement(
        "UPDATE students SET age=? WHERE id=?") {

    ps.setInt(1, 22);
    ps.setInt(2, 1);
    ps.executeUpdate();
}
```

Transaction in JDBC

```
try {
    con.setAutoCommit(false);

    // execute multiple statements

    con.commit();
} catch (Exception e) {
    con.rollback();
} finally {
    con.setAutoCommit(true);
}
```

10. SQL/JDBC Best Practices

Use parameterized queries, least-privilege DB users, connection pooling in server applications, indexes based on query patterns and transactions for multi-step consistency.

- Do not concatenate untrusted input into SQL.
- Do not store database passwords in source code.
- Use migrations/schema management in real applications.
- Close ResultSet, Statement and Connection resources.
- Prefer PreparedStatement.
- Inspect query plans for slow queries.
- Use transactions for related changes that must succeed/fail together.

PRACTICAL PROJECTS

Project 1 — Student Database

Design and query a student/course/enrollment model.

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 2 — JDBC Student Repository

Basic repository pattern.

```
public List<Student> findAll(Connection con)
    throws SQLException {

    String sql = "SELECT id, name FROM students";
    List<Student> result = new ArrayList<>();

    try (PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            result.add(new Student(
                rs.getInt("id"),
                rs.getString("name")
            ));
        }

        return result;
    }
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 3 — Transfer Transaction

Use JDBC transaction control for an atomic transfer.

```
con.setAutoCommit(false);
try {
    debitAccount(con, fromId, amount);
    creditAccount(con, toId, amount);
    con.commit();
} catch (Exception e) {
    con.rollback();
    throw e;
} finally {
    con.setAutoCommit(true);
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

INTERVIEW + VIVA REVISION

- What is a primary key?
- Foreign key?
- UNIQUE vs PRIMARY KEY?
- WHERE vs HAVING?
- INNER vs LEFT JOIN?
- UNION vs UNION ALL?
- What is a subquery?
- EXISTS?
- Normalization?
- 1NF/2NF/3NF?
- Index advantages/costs?
- ACID?
- Transaction?
- COMMIT vs ROLLBACK?
- DELETE vs TRUNCATE vs DROP?
- DDL vs DML?
- PreparedStatement vs Statement?
- What is ResultSet?
- Why close JDBC resources?
- What is SQL injection?
- How does a connection pool help?
- What is JDBC DriverManager?

FINAL CHEAT SHEET

Revise the syntax patterns, lifecycle/flow, common errors, and project architecture. The goal is not memorizing every property or method, but knowing the model well enough to build and debug applications.